University of Glasgow

Lecture 1: Course Logistics & Introduction to SSE

A WORLD TOP 100 UNIVERSITY

Dr. Yutian Tang

Email: Yutian.Tang@glasgow.ac.uk

WORLD CHANGING GLASGOW

THE SUNDAY TIMES GOOD UNIVERSITY GUIDE 2024 SCOTTISH UNIVERSITY OF THE YEAR

:Padlet for Questions

- https://padlet.com/yutiantang/sse-lecture-1-question-wall-8db3mlk4kl24hd86

Please post any questions you have during the lecture on this Padlet. I will address them at the end.

University of Glasgow

Who am I?

- Dr. Yutian Tang
  - Web: https://www.chrisyttang.org
  - Lecturer in Software Engineering
  - Email: Yutian.Tang@Glasgow.ac.uk

- Member of Software Engineering Group

- Personal research interests:
  - Software Engineering
  - System Privacy and Security
  - AI (LLM) + SE

SSE Schedule

- Lectures:
  - Every Friday 16:00 – 18:00
  - All lectures will be held at: JOSEPH BLACK: B419 MAIN Lecture Theatre
- QA session (Online & Optional):
  - Wednesday: 17:00 – 18:00
  - Zoom Link: https://uofglasgow.zoom.us/j/89729933348?pwd=EKSCxrnNTlELap1kbhOsViAtwWYEVn.1
- There are no labs for SSE.
  - Tutorials are integrated into the lecture/example content.
- Video Recording
  - Auto-enroll link: https://echo360.org.uk/collection/09ea5454-3751-4b38-9e22-85190dceced2/public

SSE Schedule'2

- Assessment Schedule:
  - A1 will be available on 30/1/2026 (Week 3) and will be due on 13/2/2026 (W5). (subject to change)
  - A2 will be available on 27/2/2026 (W7) and will be due on 13/3/2026 (W9). (subject to change)
  - The exam will take place: April/May.

- Weightings:
  - A1 is worth **10%** of your overall mark.
  - A2 is worth **10%** of your overall mark.
  - The final exam is worth **80%** of your overall mark (closed book + cheat sheet).

Attendance

- We will take attendance from lecture 2 to 10.
- NOTHING to do with your grade/mark.
- Uni policy
- If you're on a **Student Visa or a Tier 4 Visa** and you're studying a taught course, ...

School's Cheat Sheet Policy

- A standardised cheat sheet will be provided by the instructor
- Available to all students at least 2 weeks prior to the examination on course Moodle page.
- Are NOT at all expected to be comprehensive and cover every element of the course.
- There is a hard maximum of four sides of A4, ideally max. two sides (one sheet) of A4.
- In all cases, cheat sheets will be provided in a printed form to students in the exam, whether the exam itself on paper or digital.

University of Glasgow

Intended Learning Outcomes (ILOs)

By the end of this course students will be able to:

1. Describe the life cycle for developing secure software systems.
2. Apply lightweight refactoring methods to balance trade-offs between competing security, privacy and functionality quality measures in software.
3. Verify the effectiveness of a secure software design solution.
4. Explore general approaches to privacy engineering and Privacy-by-Design paradigm in software.
5. Build a simple privacy justificatory framework for justifying the extent a given software aligns with data protection regulations (e.g GDPR, HIPPA, etc.).
6. Apply secure software design principles to a range of application domains and case studies.

University of Glasgow

Recommended Texts

- You will **not** be expected to read any external textbooks for this course.
- There are several textbooks that you can read to supplement your understanding.
  - Some recommendations will be announced as we continue in this course.
- All content in the lectures can be examinable.

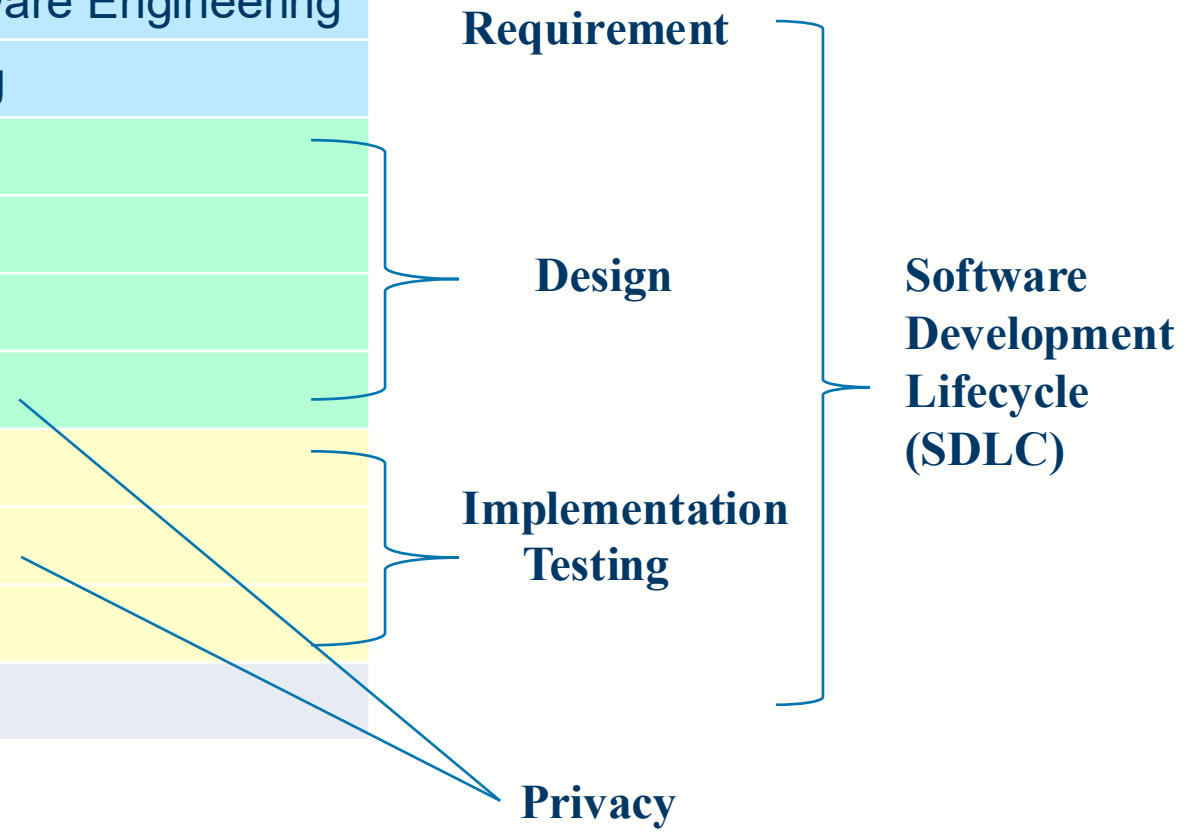
University of Glasgow

Course Timetable

Unit 1	Course Logistics & Introduction to Secured Software Engineering	16/1/2026
Unit 2	Secured SDLC & Secured Requirement Planning	23/1/2026
Unit 3	Access Control: Authentication & Authorization	30/1/2026
Unit 4	Threat Modelling	6/2/2026
Unit 5	Vulnerability Mapping	13/2/2026
Unit 6	Security Patterns	20/2/2026
Unit 7	Static Analysis & Refactoring	27/2/2026
Unit 8	Confidentiality Properties & Privacy in Software	6/3/2026
Unit 9	Secure Coding, Security Testing	13/3/2026
Unit 10	SSE Applications and Regulatory Requirements	20/3/2026

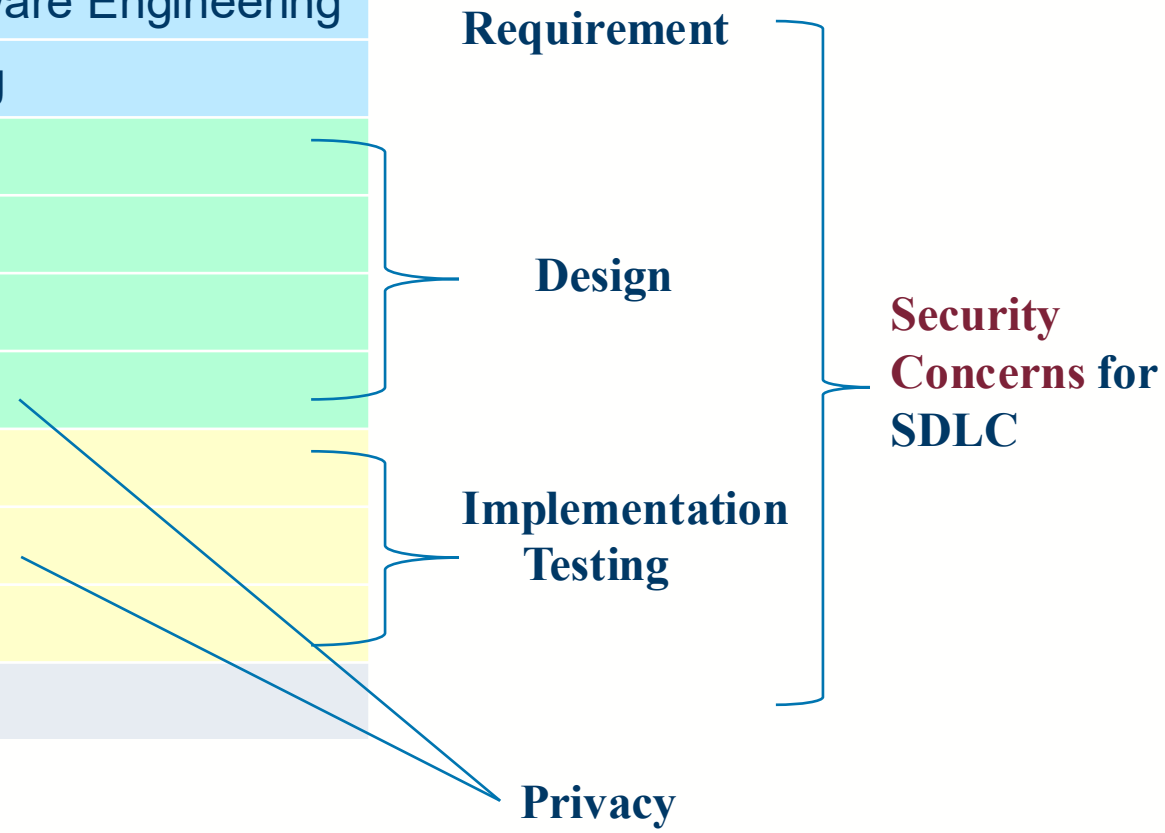
Course Timetable

Unit 1	Course Logistics & Introduction to Secured Software Engineering
Unit 2	Secured SDLC & Secured Requirement Planning
Unit 3	Access Control: Authentication & Authorization
Unit 4	Threat Modelling
Unit 5	Vulnerability Mapping
Unit 6	Security Patterns
Unit 7	Static Analysis & Refactoring
Unit 8	Confidentiality Properties & Privacy in Software
Unit 9	Secure Coding, Security Testing
Unit 10	SSE Applications and Regulatory Requirements



Course Timetable

Unit 1	Course Logistics & Introduction to Secured Software Engineering
Unit 2	Secured SDLC & Secured Requirement Planning
Unit 3	Access Control: Authentication & Authorization
Unit 4	Threat Modelling
Unit 5	Vulnerability Mapping
Unit 6	Security Patterns
Unit 7	Static Analysis & Refactoring
Unit 8	Confidentiality Properties & Privacy in Software
Unit 9	Secure Coding, Security Testing
Unit 10	SSE Applications and Regulatory Requirements



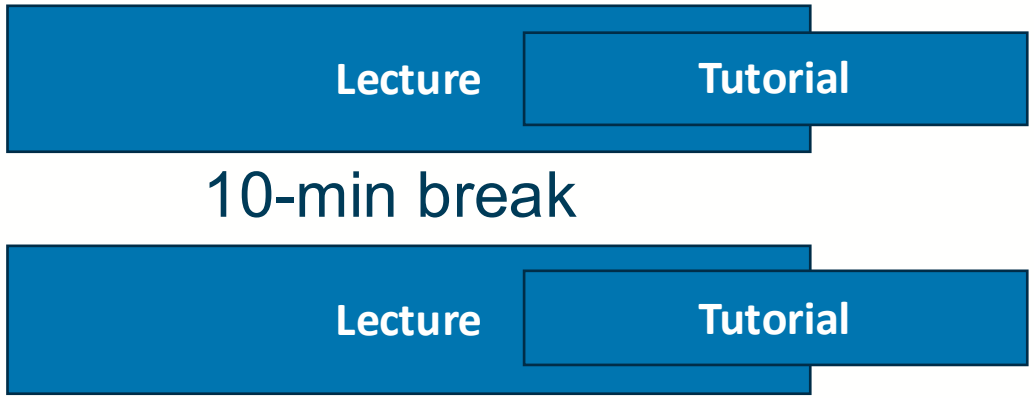
Course Timetable

Unit 1	Course Logistics & Introduction to Secured Software Engineering	ILO 1
Unit 2	Secured SDLC & Secured Requirement Planning	ILO 1,6
Unit 3	Access Control: Authentication & Authorization	ILO 3,4,6
Unit 4	Threat Modelling	ILO 3,6
Unit 5	Vulnerability Mapping	ILO 3,6
Unit 6	Security Patterns	ILO 3,6
Unit 7	Static Analysis & Refactoring	ILO 2,3,5
Unit 8	Confidentiality Properties & Privacy in Software	ILO 4
Unit 9	Secure Coding, Security Testing	ILO 2,4
Unit 10	SSE Applications and Regulatory Requirements	ILO 3,5

Resource & Support

- Resources for this course will be made available on Moodle:
- 1 hour slot every week for Q&A.
  - Online Zoom meeting
  - In-person meeting will be available by appointment. (drop me an email)
- Office hour is:
  - Wednesday: 17:00 – 18:00
  - Zoom Link: https://uofglasgow.zoom.us/j/89729933348?pwd=EKSCxrnNTlELap1kbhOsViAtwWYEVn.1
- Feel free to ask me questions on Teams

When to take a break?



University of Glasgow

Prerequisites and FAQs

- Object Oriented Software Engineering (COMPSCI2008) or its equivalent.
- Visiting students would need a firm background in object-oriented programming such as Java.

University of Glasgow

Prerequisites and FAQs

- **Q1: What programming languages are used in SSE course?**
  - While security principles can be applied to any programming language, languages like Java, Python, C++, and others are commonly used.
- **Q2: How Does Secure Software Engineering Differ From Traditional Software Engineering?**
  - Unlike traditional software engineering, which primarily focuses on functionality and performance, secure software engineering prioritizes security aspects throughout the development process. This includes threat modeling, secure coding practices, and security testing.
- **Q3: Do we need to have a solid background in Java?**
  - Yes

Prerequisites and FAQs'2

	Can Understand Java Code	Can Write Java Code
Lectures	Required	Not required
Tutorials	Required	Recommended (some programming tasks, but not assessed)
Assignments	Required	Yes
Final Exam	Required	Yes
Tools	Required	Recommended

(c) Analyze the provided "AssignmentSubmissionSystem" Java class to perform Taint Flow Analysis. Then,

- Identifying the source of tainted data;
- Tracing and listing the flow of this data through the program;
- Explain how this tainted data is used in a potentially unsafe manner.
- Refactoring the provided Java code to eliminate/mitigate the identified unsafe behavior/vulnerability (You can either refactor the provided Java code or describe it in your own words).

[6]

Prerequisites and FAQs'2

- **Q4: Do I need a background in Software Engineering to take this course?**
  - Yes, a foundational background in Software Engineering is necessary. This course builds on core Software Engineering concepts, such as the software development lifecycle, design patterns, and software architecture, to address security considerations.

Prerequisites and FAQs’3

- **Q5: Tutorial Sessions/Labs**
  - No labs
  - Tutorial Sessions:
    - Case studies;
    - Programming tasks;

University of Glasgow

Security

- What is security?

According to Dictionary.com “secure” more closely relates to Software Security
“free from or ***not exposed*** to danger or harm; safe”

- How to avoid exposure: Isolate
- Make it free from danger or harm… But how?
- Get to know, how an attacker is generally able to cause harm:
  Attackers cause problems by exploiting vulnerabilities in the system

- Vulnerability: *design flaw or implementation bugs*

University of Glasgow

In the context of Secure Software Engineering, what is a vulnerability?

- A. Any successful cyber attack
- B. A weakness caused by hardware malfunction
- C. A design flaw or implementation bug that can be exploited
- D. Any external threat actor

Join at menti.com | use code 2315 9127

University of Glasgow

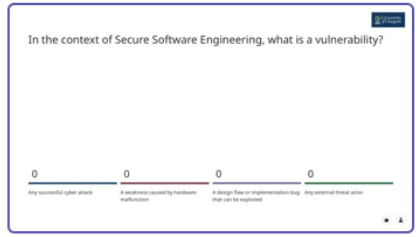
In the context of Secure Software Engineering, what is a vulnerability?

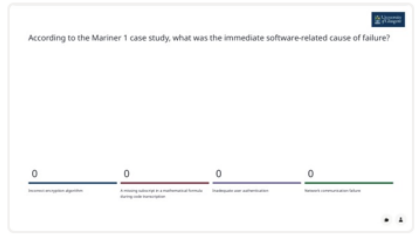
ttack	0%	0%	0%
	A weakness caused by hardware malfunction	A design flaw or implementation bug that can be exploited	Any external threat actor

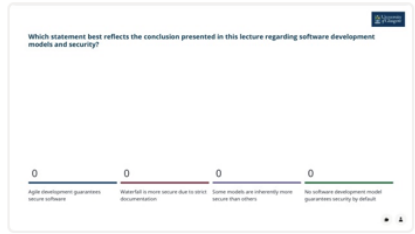
Menti

Lecture 1 Introduction

Choose a slide to present







University of Glasgow

The Prevalence of Software in Critical Sectors:

1. **Finance**: In the finance sector, software is integral for managing transactions, investments, banking services, and financial analytics.

   Automated trading systems, risk management tools, and customer relationship management (CRM) systems are just a few examples of how software underpins the financial industry.

   It ensures faster processing of data, aids in decision-making, and enhances security against frauds and cyber threats.

University of Glasgow

The Prevalence of Software in Critical Sectors:

2. **Healthcare**: In healthcare, software solutions are essential for patient record management, diagnostic procedures, treatment planning, and remote patient monitoring.

   Electronic health records (EHRs), telemedicine applications, and clinical decision support systems improve patient care, streamline operations, and facilitate medical research.

   They also enable better data management, allowing for more personalized and efficient patient care.

The Prevalence of Software in Critical Sectors:

3. **Communication**: The communication sector relies heavily on software for various services like messaging, video conferencing, and email.

   Software ensures the seamless transfer of data over networks, provides encryption for secure communication, and allows for the integration of different communication channels.

   This sector has seen exponential growth in software applications due to the increasing need for remote and digital communication, especially in the context of global connectivity and remote work trends.

The Prevalence of Software in Critical Sectors:

4. **Transportation:** Software in transportation is vital for route planning, traffic management, vehicle tracking, and safety systems.

   It plays a critical role in the functioning of GPS systems, flight operation systems in aviation, and logistics management in shipping.

   The advancement of software has also paved the way for autonomous vehicles and intelligent transportation systems, enhancing efficiency, safety, and sustainability in the transportation sector.

University of Glasgow

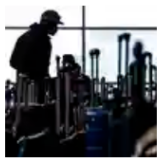
It is always hard to make it safe

University of Glasgow

The U.S. Department of Transportation is investigating the Southwest Airlines holiday travel debacle, which left thousands of travelers stranded for days. The investigation comes as the airline reported a $220 million loss last quarter and further losses in the first quarter.

Southwest canceled more than 16,700 flights over several days in late December. While a massive winter storm caused the initial cancellations, the company's outdated software systems turned what should have been a normal problem into a snowballing disaster that lasted for days after other airlines had resumed their usual operations.

The department's investigation will look into whether Southwest made unrealistic flight schedules, "which under federal law is considered an unfair and deceptive practice," according to a department spokesperson.



BUSINESS

5 things to know about Southwest's disastrous meltdown

"DOT has made clear to Southwest that it must provide timely refunds and reimbursements and will hold Southwest accountable if it fails to do so," the department spokesperson said.

Ref: https://www.npr.org/2023/01/26/1151667801/southwest-airlines-investigation-losses-holiday-travel-cancellations

The Mariner 1 (1962) spacecraft headed for Venus diverted from its intended path after 293 seconds of liftoff. The mission was completed by Mariner 2 which launched 5 weeks later.

This is a combination of two failures – an antenna hardware failure and an onboard guidance system software failure. The guidance antenna performed below its specification. So, the spacecraft had to rely on its onboard guidance system, which had a bug in it.

A programmer incorrectly transcribed a formula into computer code, missing a single subscript bar, which was meant for nth smoothed value of the time derivative of a radius R.

The smallest error can lead to the largest of failures.

Ref: https://airandspace.si.edu/multimedia-gallery/mariner2launchjpg

University of Glasgow

According to the Mariner 1 case study, what was the immediate software-related cause of failure?

- A. Incorrect encryption algorithm
- B. A missing subscript in a mathematical formula during code transcription
- C. Inadequate user authentication
- D. Network communication failure

Join at menti.com | use code 2315 9127

University of Glasgow

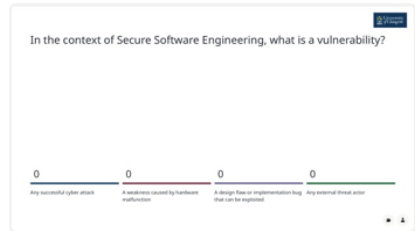
According to the Mariner 1 case study, what was the immediate software-related cause of failure?

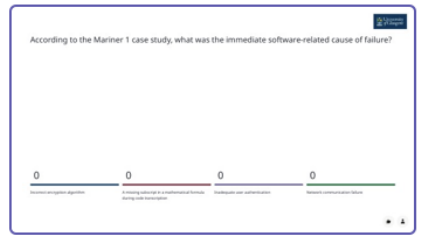
hm	0%	0%	0%
	A missing subscript in a mathematical formula during code transcription	Inadequate user authentication	Network communication failure

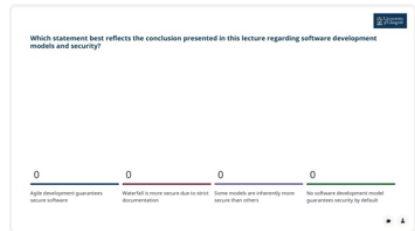
Menti

Lecture 1 Introduction

Choose a slide to present







University of Glasgow

So, It is always hard to make it safe

University of Glasgow

Consequences

- Consequences of Insecure Software:
  - **Data Breaches**: Unauthorized access to sensitive information.
  - **Financial Losses**: Costs incurred from cyber attacks, fraud, and data theft.
  - **Reputational Damage**: Loss of customer trust and brand integrity.
  - **Operational Disruptions**: Interruptions to business operations and services.
  - **Legal and Compliance Issues**: Penalties and legal actions due to non-compliance with regulations.
  - …

University of Glasgow

What is Design?

University of Glasgow

What is Design in Software Engineering?

- Software design as the activity of defining a software system's architecture, components, interfaces, and other characteristics to satisfy given requirements. (Guide to the Software Engineering Body of Knowledge)

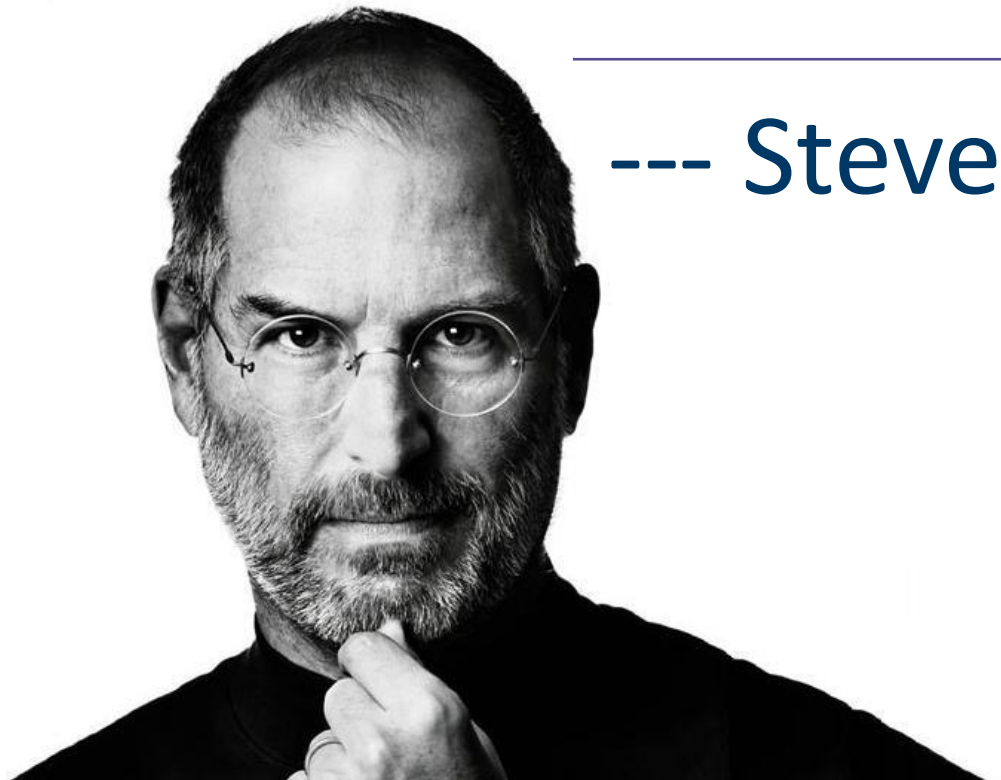
- Design marks the transition from understanding the problem to inventing a solution for it.

- This transformation is crucial for guiding programmers in coding and implementation.

University of Glasgow

"Design is not just what it looks like and feels like. Design is how it works"

--- Steve Jobs



Process of Software Design (Top-down Design Strategy)

- Top-down design strategy (start with the architecture)
- **Define** Top-down design:
  - A hierarchical approach to software development where the system is broken down from the general to the specific.

- **Key Principles**:
  1. Begin with identifying the core functionalities of the system.
  2. Break down these functionalities into smaller, more manageable components.
  3. Focus on the high-level design and structure before moving on to detailed implementation.

University of Glasgow

Process of Software Design (Top-down Design Strategy)

- Steps of Top-down design:
  1. Identify system-wide design decisions that provide overall guidance.
  2. Decompose the system into subsystems or modules
  3. Specify the role of each module and its interface with others.
  4. Iterate and refine the design, working through layers of detail.
     - such as:
     - *The format of data items.*
     - *The individual algorithms that will be used.*
     - *How different algorithms communicate with each other.*

Process of Software Design (Top-down Design Strategy)

- Top-down Design Strategy (start with the architecture)
- Advantages:
  - Facilitates clear understanding at each level of design.
  - Simplifies complex systems by breaking them into more digestible parts.
  - Encourages modularity and reusable code structures.

Process of Software Design (Bottom-up Design Strategy)

- Bottom-up design (start with low level utilities)
- **Definition**:
  - A software development approach that starts with specific, detailed components or modules, which are then integrated to form more complex systems.

- Key **Characteristics**:
  - Emphasis on creating *reusable code* modules.
  - Focus on detailed solutions and individual components before integrating into the whole system.
  - Often used in conjunction with top-down design for a balanced approach.

Process of Software Design (Bottom-up Design Strategy)

- **Steps in Bottom-Up Design:**
  1. Development starts with detailed-level coding and design.
  2. Modules are developed and tested independently.
  3. Integration of modules to form subsystems, then progressively into the complete system.

Advantages:

- Allows for parallel development and testing.
- Facilitates flexibility and adaptability in the design process.

Process of Software Design (Mix of top-down and bottom-up)

- Overview of Hybrid Approach:
  - Utilizes the strengths of both top-down and bottom-up methodologies to enhance software design.
- **Process**:
  - Start with a high-level design outline (top-down) to establish the overall structure and main components.
  - Develop individual modules and components in detail (bottom-up).
  - Iteratively integrate and refine components within the overarching design framework.
- **Benefits**:
  - Balances big-picture planning with detailed development.
  - Facilitates flexibility and comprehensive system view.
  - Enhances adaptability to changes and iterative improvements.

University of Glasgow

Process of Software Design (Mix of top-down and bottom-up)

Mix of top-down and bottom-up:

- Top-down design is almost always needed to give the system a good structure.
- Bottom-up design is normally useful so that reusable components can be created.

Case Study: Hybrid Design in Action

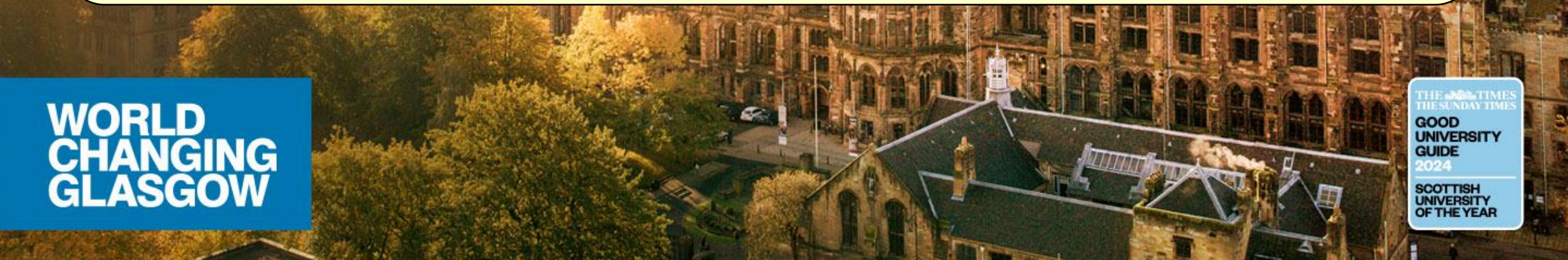
- Example Scenario: Development of an E-commerce Platform.
  - Initial top-down phase for outlining user experience, core features, and architecture.
  - Bottom-up development for individual features like payment processing, product catalog, user authentication.
  - Continuous integration of these modules into the planned architecture.

Lecture 1: Course Logistics & Introduction to SSE

Dr. Yutian Tang

Email: Yutian.Tang@glasgow.ac.uk

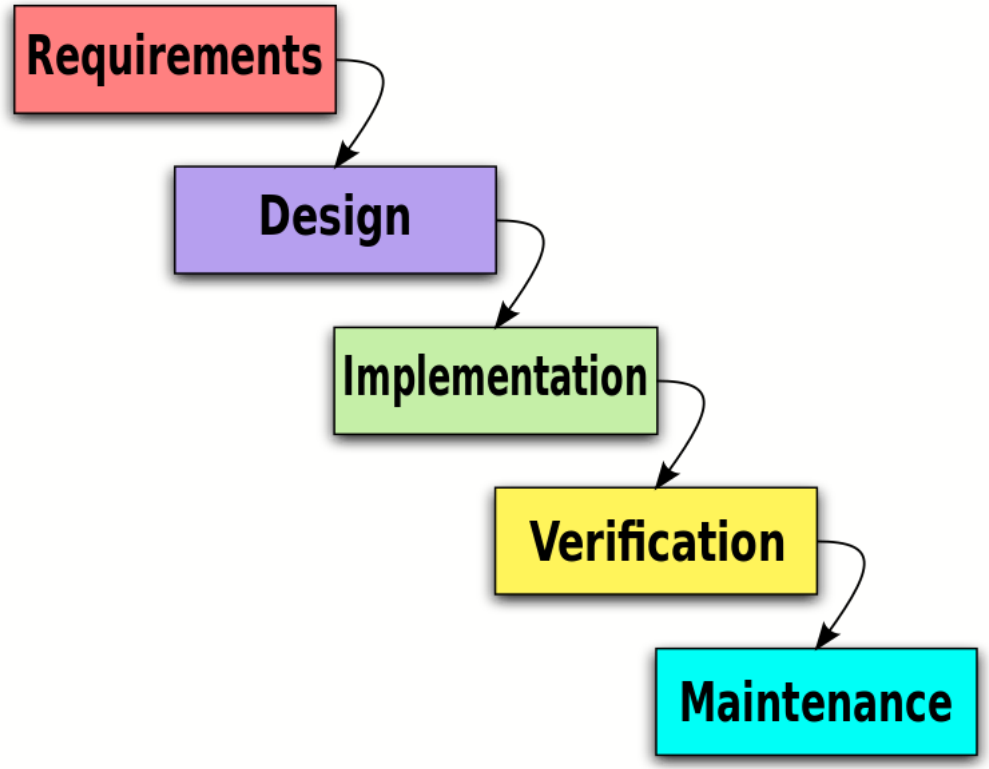
We are resuming at :



Can Current Software Development Methods Produce Secure Software?

Software Development Models

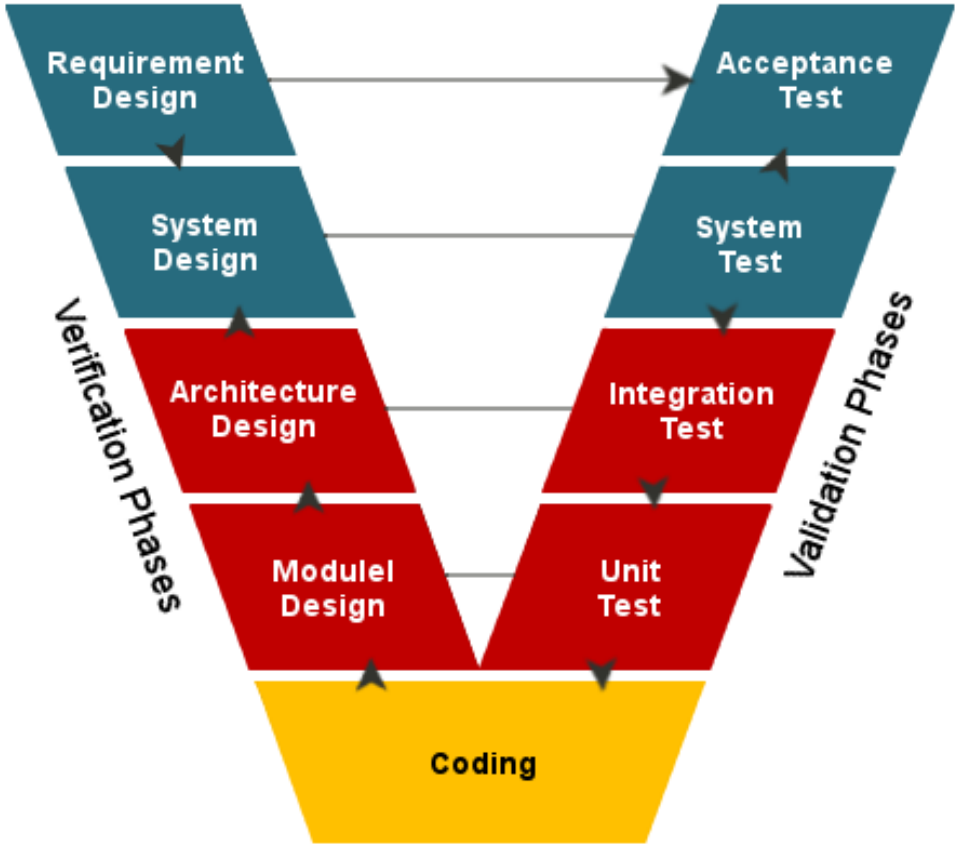
- **Waterfall Model:**
  - *Description*: Linear, sequential approach with cascade movement through stages: requirement, design, implementation, verification, maintenance.
  - *Characteristics*: Each stage has specific deliverables and is strictly documented. No overlapping or revisiting of stages.



https://eternalsunshineoftheismind.wordpress.com/2013/02/28/the-waterfall-model-a-traditional-method-of-software-design/

Software Development Models'2

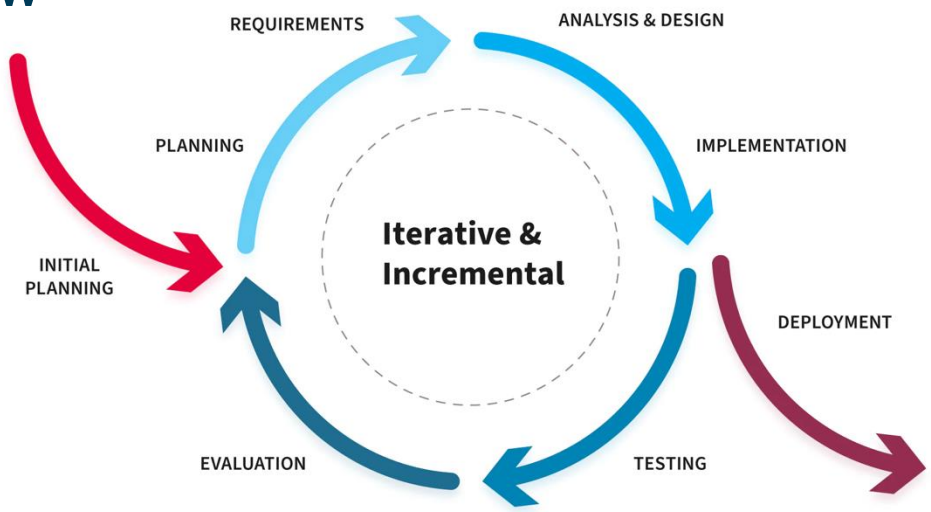
- **V-model (Validation and Verification Model)**
  - *Description*: Linear model, each development stage has a corresponding testing activity.
  - *Characteristics*: High quality control; expensive and time-consuming. Changes during development are costly



https://softwareengineering.stackexchange.com/questions/394860/does-the-v-model-have-a-verification-and-validation-stage-for-every-stage

Software Development Models'3

- **Incremental and Iterative Model (IIM)**
  - *Description*: Development splits into iterations; new modules are added in each iteration.
  - *Characteristics*: Allows for some changes in requirements during development. Works well for evolving software.



- **Spiral-model**
  - *Description*: Focus on risk assessment with iterations involving planning, risk analysis, prototype creation.
  - *Characteristics*: Customer involvement in exploration and review stages. Suited for projects with unclear or innovative requirements.

https://nix-united.com/blog/software-development-life-cycle-nix-approach-to-sdlc/

University of Glasgow

Agile Software Development

- Agile Development/Programming
- They attempt to reduce the overall risk of a software development project by building software in very rapid iterations.
- Each rapid iteration is often called sprint.
- These short turnarounds potentially allow for better customer feedback and interaction, time management, and schedule prediction
- It cannot ensure security.

University of Glasgow

Common Criteria

- The Common Criteria (CC), also referred to as ISO/IEC 15408 (Common Criteria 2006).
- It is an international standard for computer security to assess the presence and assurance of security features.
- Its goal is to allow users to define their security requirements, have developers specify the security attributes of their products, and, finally, allow third-party evaluators to determine whether the products meet the stated claims.

University of Glasgow

Common Criteria'2

- What CC does provide is evidence that security-related features perform as expected
  - For example, *if a product provides an access control mechanism to objects under its control, a CC evaluation would provide assurance that…*

- BUT, it is possible that some features in the system are vulnerable, which can be comprised.

- That is why CC cannot guarantee security.

University of Glasgow

Which statement best reflects the conclusion presented in this lecture regarding software development models and security?

- A. Agile development guarantees secure software
- B. Waterfall is more secure due to strict documentation
- C. Some models are inherently more secure than others
- D. No software development model guarantees security by default

Join at menti.com | use code 2315 9127

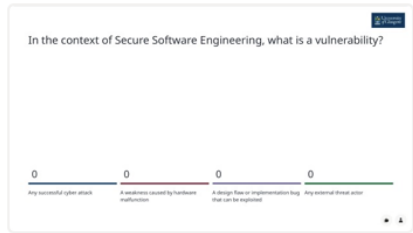
University of Glasgow

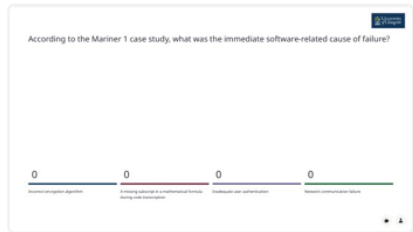
Which statement best reflects the conclusion presented in this lecture regarding software development models and security?

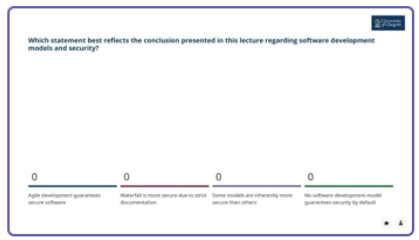
ırantees	Waterfall is more secure due to strict documentation	Some models are inherently more secure than others	No software development model guarantees security by default
	0%	0%	0%

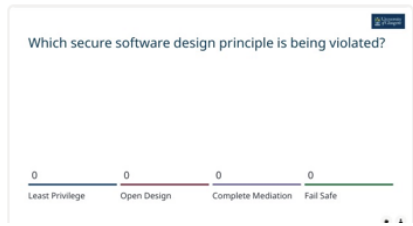
Menti

Lecture 1 Introduction









University of Glasgow

Software Development Models'4

- ….

- There is no evidence whatsoever that any of these methods create more secure software than another internal development method.

- In fact, many of these software development methods make no mention of the word "security" in their documentation

Current Software Development Methods MAY NOT Produce Secure Software.

The Principles of Secure Software Design

Principles of Secure Software Design

1. Defence in Depth
2. Fail Safe
3. Least Privilege
4. Separation of Duties
5. Economy of Mechanism
6. Complete Mediation
7. Open Design
8. Least Common Mechanism
9. Psychological Acceptability
10. Weakest Link
11. Leveraging Existing Components

https://owasp.org/www-project-developer-guide/draft/

University of Glasgow

Principles of Secure Software Design'2

1. Defence in Depth

Def: This involves multiple layers of security controls to protect an organization's assets. If one layer fails, others still provide protection.

Eg.: Implementing both a firewall and an intrusion detection system (IDS) in a network. If an attacker breaches the firewall, the IDS serves as an additional layer of protection.

University of Glasgow

Principles of Secure Software Design'2

2. Fail Safe

Def: When a system error occurs, the system should safeguard the secrecy, accuracy, and availability of information. (Revert to a safe condition in the event of a breakdown or malfunction)

Eg.: An application automatically logging out a user after a period of inactivity, thereby preventing unauthorized access if the user forgets to log out.

University of Glasgow

Principles of Secure Software Design’3

3. **Least Privilege**

   Def: Providing individuals or processes with the minimal access rights required to accomplish their tasks.

   Eg.: A database administrator having only the necessary permissions to manage the database, but not the permissions to modify user accounts or access unrelated systems.

University of Glasgow

Principles of Secure Software Design’4

4. **Separation of Duties**

   Def: Separation of duties is a fundamental principle of internal control in business and organizations.

   It is a system of checks and balances that ensures that no single individual has control over all aspects of a transaction.

   Eg.: A practical application might be in financial processes, where one employee is responsible for recording transactions, and another is responsible for approving them.

University of Glasgow

Principles of Secure Software Design’5

5. **Economy of Mechanism**

   Def.: Keeping systems simple and small, thus reducing the chances of security vulnerabilities.

   Eg.: Minimizing the amount of code used in a program, reducing its attack surface.

University of Glasgow

Principles of Secure Software Design'6

6. Complete Mediation

Def.: Every access request must be checked against the access control mechanism.

Eg.: Each file access request is validated for permissions every time, without relying on cached permissions.

University of Glasgow

Principles of Secure Software Design’7

7. Open Design

Def: The open design security principle states that the implementation details of the design should be independent of the design itself, allowing the design to remain open while the implementation can be kept secret.

The security of a system should not rely on the secrecy of its design

Eg.: When software is architected using the open design concept, the review of the design itself will not result in the compromise of the safeguards in the software.

University of Glasgow

Principles of Secure Software Design’8

8. Least Common Mechanism

Def: Minimizing the amount of mechanisms shared by different users or processes.

The goal is to reduce the chances of a security breach affecting multiple users simultaneously

Eg.: In a multi-tenant cloud environment, ensuring that each tenant's data is stored and processed in isolated environments to prevent one tenant's activities from affecting another's.

University of Glasgow

Principles of Secure Software Design’9

9. Psychological Acceptability

Def: A security principle that aims at maximizing the usage and adoption of the security functionality in the software by ensuring that the security functionality is easy to use and at the same time transparent to the user.

Eg.: The use of fingerprint scanners on smartphones for authentication. This method offers strong security but is also user-friendly and quick, encouraging widespread adoption by users.

University of Glasgow

Principles of Secure Software Design'10

10. Weakest Link

Def: This security principle states that the resiliency of your software against hacker attempts will depend heavily on the protection of its weakest components, be it the code, service or an interface.

Eg.: In a secure web application, the presence of a third-party plugin with outdated security makes it vulnerable to attacks

University of Glasgow

Principles of Secure Software Design'11

11. Leveraging Existing Components

Def.: This is a security principle that focuses on ensuring that the attack surface is not increased and no new vulnerabilities are introduced by promoting the reuse of existing software components, code and functionality.

Eg.: Instead of writing their own encryption algorithms, developers often use existing libraries like OpenSSL or Microsoft's CryptoAPI.

University of Glasgow

Case Study

- A system validates all user inputs, but an attacker is still able to bypass validation by sending data through an undocumented internal interface.
- Which secure software design principle is being violated?
- A. Least Privilege
- B. Open Design
- C. Complete Mediation
- D. Fail Safe

Join at menti.com | use code 2315 9127

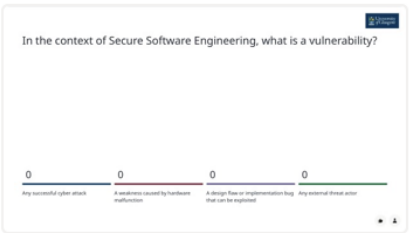
Which secure software design principle is being violated?

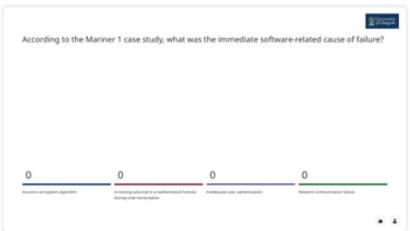
0	0	0
Open Design	Complete Mediation	Fail Safe

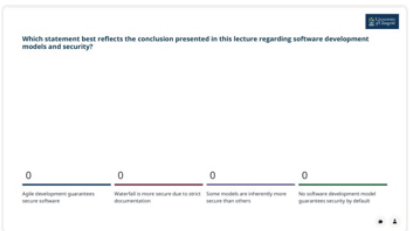
Menti

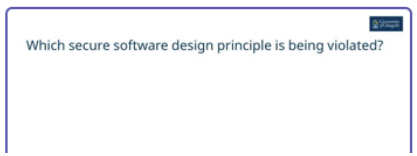
Lecture 1 Introduction

Choose a slide to present









:Padlet for Questions

- https://padlet.com/yutiantang/sse-lecture-1-question-wall-8db3mlk4kl24hd86

Please post any questions you have during the lecture on this Padlet. I will address them at the end.

Acknowledgement and References

- Secure Programming with Static Analysis;
- https://www.ncsc.gov.uk/collection/developers-collection;
- https://owasp.org/www-project-developer-guide/draft/
- https://www.npr.org/2023/01/26/1151667801/southwest-airlines-investigation-losses-holiday-travel-cancellations
- https://www.oberlo.com/statistics/how-many-people-have-smartphones
- https://www.bankmycell.com/blog/how-many-phones-are-in-the-world
- https://iot-analytics.com/number-connected-iot-devices/
- https://eternalsunshineoftheismind.wordpress.com/2013/02/28/the-waterfall-model-a-traditional-method-of-software-design/

Acknowledgement and References (2)

- https://softwareengineering.stackexchange.com/questions/394860/does-the-v-model-have-a-verification-and-validation-stage-for-every-stage
- https://nix-united.com/blog/software-development-life-cycle-nix-approach-to-sdlc/
- https://unsplash.com/s/photos/free-image
- Designing Secure Software: A Guide for Developers
- C. M. M. Bezerra, S. C. B. Sampaio, and M. L. M. Marinho, “Secure Agile Software Development: Policies and Practices for Agile Teams,” in International Conference on the Quality of Information and Communications Technology, Cham, 2020, pp. 343-357.
- D. A. Barbosa, and S. Sampaio, “Guide to the support for the enhancement of security measures in agile projects,” in 2015 6th Brazilian Workshop on Agile Methods (WBMA), 2015, pp. 25-31.
- M. Gondree, Z. N. Peterson, and T. Denning, “Security through play,” IEEE Security & Privacy, vol. 11, no. 3, pp. 64-67, 2013.


University of Glasgow
Thank you!
Any Questions?